Conversational Speaking Guide: OpenText & Google Drive Integration (USV)

(Best-Practice & 3+ Years Experience Architecture Edition)

This guide provides conversational scripts and structured answers to help you present the **3_USV** (P02_DocXIntegration_USV) project in interviews at a **3+ years of experience** developer level.

1. The 60-Second "Elevator Pitch" (Overview)

Question: "Tell me about your Google Drive / OpenText integration project."

💡 The Best-Practice Response:

> "At Supai Infotech, I designed and developed a middleware integration gateway connecting OpenText Content Server (OTCS) with the Google Workspace API.
>
> The business challenge was that while OTCS was our secure, audited repository of record, business users needed Google Workspace's real-time collaborative editing features. To solve this, I built a checkout/check-in middleware in **Spring Boot** running inside a **Docker** environment on **Kubernetes**.
>
> From an engineering perspective, the system implements the **Orchestration Saga Pattern** to manage document lifecycles:

> - It enforces concurrency control via **OTCS reservation locks** to prevent dual edits.
> - It manages secure server-to-server calls via **Google Service Accounts and OAuth 2.0**.
> - It maintains state synchronization by mapping GDrive Document IDs and Editor IDs directly into OTCS category metadata.
> - It treats Google Drive as an ephemeral workspace, dynamically granting reader/writer roles using the **Google Permissions API** and deleting files upon check-in to prevent storage leaks and comply with corporate data security standards."

2. High-Level Design (HLD) & Technical Architecture Pitch

Question: "Can you walk me through the integration architecture and components?"

💡 The Best-Practice Response:

> "The architecture is built on Spring Boot, containerized with Docker, and deployed on Kubernetes.
>
> We used an **Orchestration Pattern** to coordinate transactions across three main layers: the API boundary, our core orchestration service, and the client wrapper layer.
>
> 1. **Authentication**: To optimize Google API token exchange, I implemented a token caching layer using **Redis**. We cache the Google access tokens with a 50-minute TTL, matching the OAuth token lifecycle. This reduced our token exchange overhead by 95%.
> 2. **State and Locking**: We use OTCS's native reservation lock to lock the document locally. During this edit window, we save the GDrive file ID and editor ID directly into the document's OTCS category attributes. If other users attempt to edit the file, the orchestrator detects the lock, queries the category metadata, and calls the Google Permissions API to grant the secondary user reader-only (view) access to the active Google Doc.
> 3. **Observability**: I renamed execution threads contextually using the target Document ID. Because we stream our logs to **Splunk / ELK**, this custom thread naming acts as a Correlation ID, allowing us to trace a document's entire lifecycle through log metrics instantly."

3. Explaining Resilience, Testing, & Production Practices (3+ Years Level)

Q1: How did you handle network latency or API timeouts when communicating with external Google or OpenText services?

💡 **The Best-Practice Response:**

> "We implemented resilience at the client integration layer:
>
> 1. **HTTP Connection Pooling**: We configured `RestTemplate` with an Apache `HttpClient` connection pool (`PoolingHttpClientConnectionManager`) to reuse TCP connections, reducing the handshaking latency and socket exhaustion under heavy loads.
> 2. **Resilience4j Retries**: For outbound REST calls to OpenText and Google APIs, we wrapped the HTTP calls in Resilience4j retries with exponential backoff (e.g., 3 retries, starting at 1 second, doubling on each failure) to handle transient blips.
> 3. **Actuator Health Checks**: In production, Kubernetes uses Spring Boot Actuator's `/actuator/health` endpoint for liveness and readiness probes, ensuring traffic is only routed to healthy gateway pods."

Q2: How did you test this microservice, and what was your target coverage?

💡 **The Best-Practice Response:**

> "I wrote comprehensive unit and integration tests to ensure code quality:
>
> - **Unit Testing**: I used **JUnit 5** and **Mockito** to mock downstream OpenText REST APIs and Google Drive client calls, verifying that edge cases (like unauthorized check-ins or expired Google tokens) were handled properly.
> - **MockRestServiceServer**: I utilized Spring's `MockRestServiceServer` to test my RestTemplate configuration and mock exact REST responses.
> - **CI/CD Integration**: We set up a **GitHub Actions CI/CD pipeline** that runs the test suites, scans for code smells via **SonarQube**, and builds the Docker image only if code coverage exceeds our team's **80% threshold**."

4. Summary Cheat Sheet of Key Architectural Concepts

Feature	Baseline Approach	💡 Best-Practice Explanation (What to Say)
Concurrency Control	Simple file copying	**OTCS Reservation Locks** (Prevents dual edits and enforces repository integrity)
Token Optimization	Fetching OAuth token on each request	**Redis Caching with TTL** (Caches tokens for 50 mins, reducing roundtrips by 95%)
Google Drive Cleanup	Leaving files in GDrive	**Transient Workspace Strategy** (Strict post-merge deletions to satisfy security audits and prevent storage leaks)
State Logging	Default thread names	**Contextual Thread Renaming** (Renames threads to `otcsDocId` to act as a correlation ID in Splunk/ELK)
Resilience	Standard try/catch	**Resilience4j + Connection Pooling** (Auto-recovers from transient network blips and prevents socket exhaustion)
CI/CD & DevOps	Manual build/push	**GitHub Actions + Docker + Kubernetes** (Automated builds, SonarQube quality gates, Kubernetes rolling updates)